

图像处理基础课程课后作业九

建智 2201 谢天赐 20221082044

9.1. 1)

均衡化直方图的具体步骤：

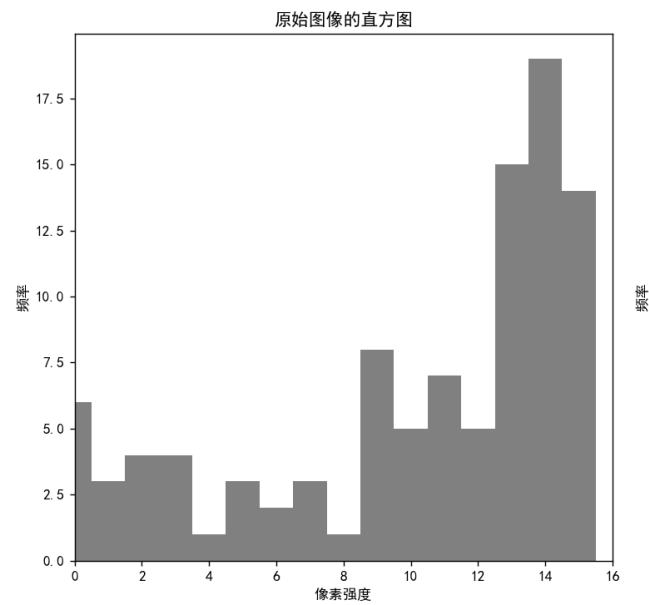
步骤 1：计算原始图像的直方图

首先，我们需要计算给定的 10x10 像素图像的直方图。直方图显示了每个灰度值（0-15）出现的次数。给定的图像数据如下：

1	2	9	12	13	12	14	14	15	15
3	9	13	13	13	14	15	15	11	8
6	11	13	14	15	13	10	6	5	3
10	14	13	14	11	3	1	0	0	0
11	13	14	11	2	0	3	9	7	5
13	15	13	9	2	7	10	14	14	12
13	12	11	2	5	10	14	14	13	14
13	14	9	0	9	12	14	15	15	15
14	10	9	1	9	14	15	15	15	14
13	11	7	0	4	14	15	15	14	13

计算每个灰度值的出现次数：

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
6	3	4	4	1	3	2	3	1	8	5	7	5	15	19	14



步骤 2：计算累积频率分布

累积频率分布是每个灰度值之前所有灰度值频率的总和。

累积频率：

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
6	9	13	17	18	21	23	26	27	35	40	47	52	67	86	100

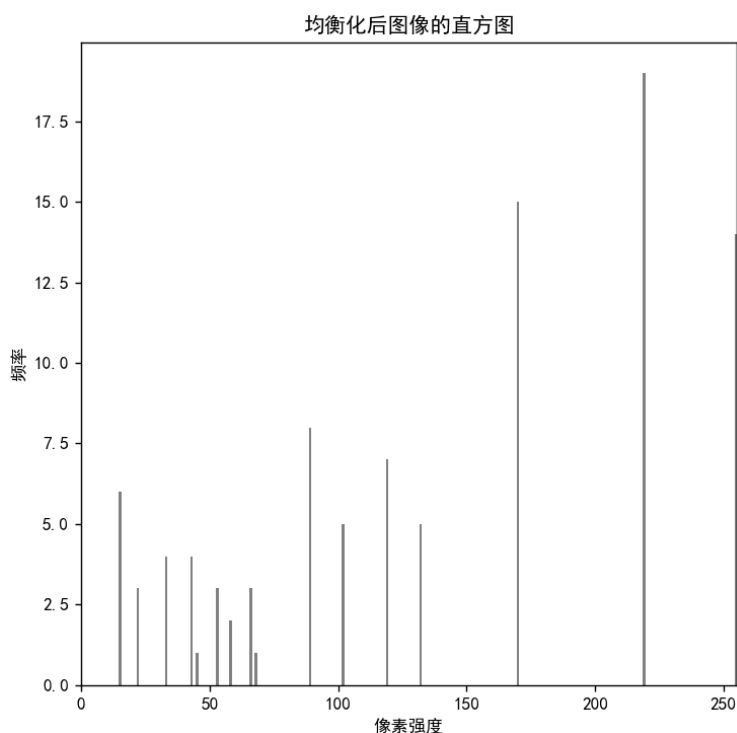
步骤 3：应用直方图均衡化

为了均衡化直方图，我们需要将原始灰度值映射到新的灰度值，使得新的直方图更加均匀。

新灰度值可以通过以下公式计算：

$$\text{新灰度值} = \text{累计频率} \times 255$$

计算新的灰度值：



步骤 4：修改图像的灰度值使用计算出的新灰度值，我们替换原始图像中的每个像素值。

9.1. 2)

修改像素点的灰度值：

```

1 22, 33, 89, 132, 170, 132, 219, 219, 255, 255
2 43, 89, 170, 170, 170, 219, 255, 255, 119, 68
3 58, 119, 170, 219, 255, 170, 102, 58, 53, 43
4 102, 219, 170, 219, 119, 43, 22, 15, 15, 15
5 119, 170, 219, 119, 33, 15, 43, 89, 66, 53
6 170, 255, 170, 89, 33, 66, 102, 219, 219, 132
7 170, 132, 119, 33, 53, 102, 219, 219, 170, 219
8 170, 219, 89, 15, 89, 132, 219, 255, 255, 255
9 219, 102, 89, 22, 89, 219, 255, 255, 255, 219
0 170, 119, 66, 15, 45, 219, 255, 255, 219, 170
1

```

	0	1	2	3	4	5	6	7	8	9
0	22	33	89	132	170	132	219	219	255	255
1	43	89	170	170	170	219	255	255	119	68
2	58	119	170	219	255	170	102	58	53	43
3	102	219	170	219	119	43	22	15	15	15
4	119	170	219	119	33	15	43	89	66	53
5	170	255	170	89	33	66	102	219	219	132
6	170	132	119	33	53	102	219	219	170	219
7	170	219	89	15	89	132	219	255	255	255
8	219	102	89	22	89	219	255	255	255	219
9	170	119	66	15	45	219	255	255	219	170

9.1. 3)

自编程序完成 1)、2)：

部分代码如下所示：（末尾附有完整代码）

```

# 将数据转换为numpy数组
image = np.array(data)

# 计算原始图像的直方图
hist, bins = np.histogram(image, bins=16, range=(0, 16))

# 计算累积频率分布
cdf = hist.cumsum()
cdf_normalized = cdf * 255 / cdf.max()

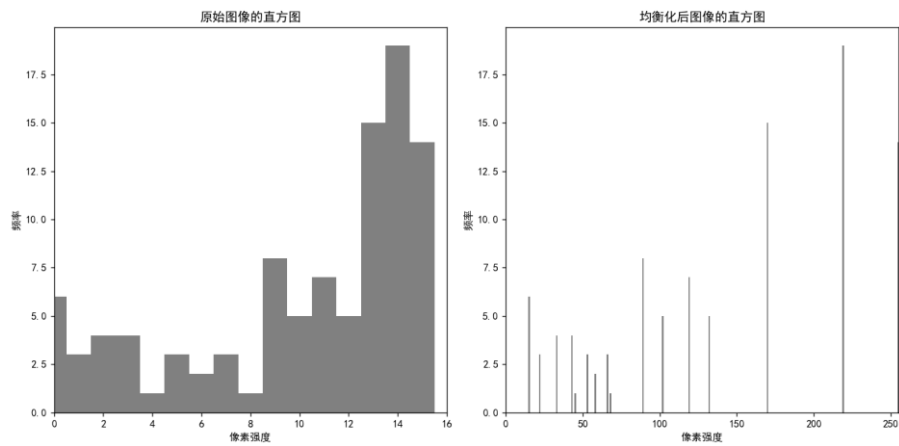
# 映射到新的灰度值
equalized_cdf = np.interp(image.flatten(), bins[:-1], cdf_normalized)
equalized_image = equalized_cdf.reshape(image.shape).astype(np.uint8)

# 计算均衡化后的图像的直方图
equalized_hist, equalized_bins = np.histogram(equalized_image, bins=256, range=(0, 256))

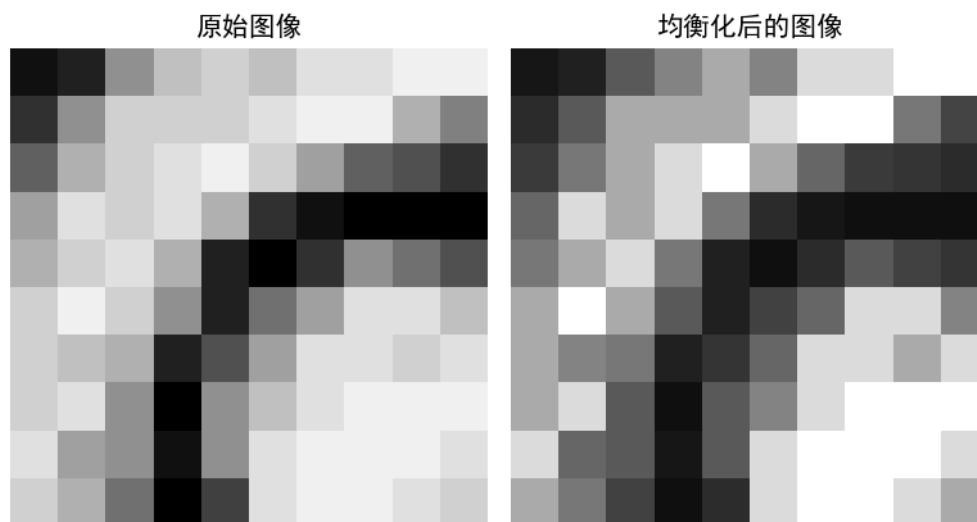
# 绘制原始图像的直方图
plt.figure(figsize=(12, 6))
plt.subplot(1, 2, 1)
plt.title("原始图像的直方图")
plt.xlabel("像素强度")
plt.ylabel("频率")
plt.xlim([0, 16]) # 设置x轴的范围
plt.bar(bins[:-1], hist, width=1, color='gray') # 绘制直方图

```

生成均衡前后的直方图对比：



原始图像均衡前后对比：



代码：

```
import numpy as np
import matplotlib.pyplot as plt

# 设置 matplotlib 的字体为支持中文的字体，例如使用黑体
plt.rcParams['font.sans-serif'] = ['SimHei']
```

读取原始数据

```
data = [
    [1, 2, 9, 12, 13, 12, 14, 14, 15, 15],
    [3, 9, 13, 13, 13, 14, 15, 15, 11, 8],
    [6, 11, 13, 14, 15, 13, 10, 6, 5, 3],
    [10, 14, 13, 14, 11, 3, 1, 0, 0, 0],
    [11, 13, 14, 11, 2, 0, 3, 9, 7, 5],
    [13, 15, 13, 9, 2, 7, 10, 14, 14, 12],
    [13, 12, 11, 2, 5, 10, 14, 14, 13, 14],
    [13, 14, 9, 0, 9, 12, 14, 15, 15, 15],
    [14, 10, 9, 1, 9, 14, 15, 15, 15, 14],
```

```

    [13, 11, 7, 0, 4, 14, 15, 15, 14, 13]
]

# 将数据转换为 numpy 数组
image = np.array(data)

# 计算原始图像的直方图
hist, bins = np.histogram(image, bins=16, range=(0, 16))

# 计算累积频率分布
cdf = hist.cumsum()
cdf_normalized = cdf * 255 / cdf.max()

# 映射到新的灰度值
equalized_cdf = np.interp(image.flatten(), bins[:-1], cdf_normalized)
equalized_image = equalized_cdf.reshape(image.shape).astype(np.uint8)

# 计算均衡化后的图像的直方图
equalized_hist, equalized_bins = np.histogram(equalized_image, bins=256,
range=(0, 256))

# 绘制原始图像的直方图
plt.figure(figsize=(12, 6))
plt.subplot(1, 2, 1)
plt.title("原始图像的直方图")
plt.xlabel("像素强度")
plt.ylabel("频率")
plt.xlim([0, 16]) # 设置 x 轴的范围
plt.bar(bins[:-1], hist, width=1, color='gray') # 绘制直方图

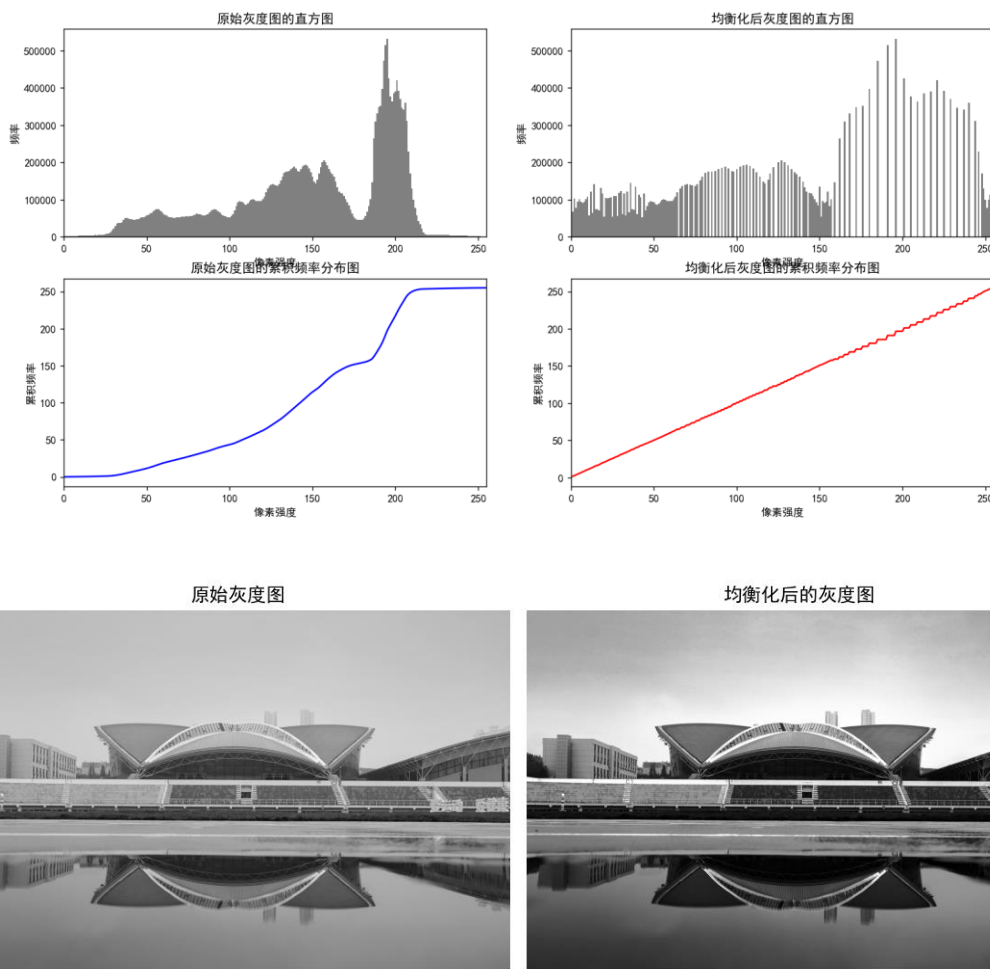
# 绘制均衡化后的图像的直方图
plt.subplot(1, 2, 2)
plt.title("均衡化后图像的直方图")
plt.xlabel("像素强度")
plt.ylabel("频率")
plt.xlim([0, 255]) # 设置 x 轴的范围
plt.bar(equalized_bins[:-1], equalized_hist, width=1, color='gray') # 绘制直方图

plt.tight_layout()
plt.show()

```

9.1. 4)

自编程序完成 1)、2):



完整代码:

```
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image

# 设置 matplotlib 的字体为支持中文的字体，例如使用黑体
plt.rcParams['font.sans-serif'] = ['SimHei'] # 'SimHei' 是黑体的字体名称
plt.rcParams['axes.unicode_minus'] = False # 正确显示负号

# 读取彩色图像并转换为灰度图
color_image = Image.open('gym.jpg').convert('L')
gray_image = np.array(color_image)

# 计算原始灰度图的直方图
hist, bins = np.histogram(gray_image, bins=256, range=(0, 256))

# 计算累积频率分布
cdf = hist.cumsum()
```

```

cdf_normalized = cdf * 255 / cdf.max()

# 映射到新的灰度值
equalized_cdf = np.interp(gray_image.flatten(), bins[:-1], cdf_normalized)
equalized_image = equalized_cdf.reshape(gray_image.shape).astype(np.uint8)

# 计算均衡化后的图像的直方图
equalized_hist, equalized_bins = np.histogram(equalized_image, bins=256,
range=(0, 256))

# 绘制原始灰度图的直方图
plt.figure(figsize=(12, 8))
plt.subplot(2, 2, 1)
plt.title("原始灰度图的直方图")
plt.xlabel("像素强度")
plt.ylabel("频率")
plt.xlim([0, 255])
plt.bar(bins[:-1], hist, width=1, color='gray')

# 绘制均衡化后灰度图的直方图
plt.subplot(2, 2, 2)
plt.title("均衡化后灰度图的直方图")
plt.xlabel("像素强度")
plt.ylabel("频率")
plt.xlim([0, 255])
plt.bar(equalized_bins[:-1], equalized_hist, width=1, color='gray')

# 绘制原始灰度图的累积频率分布图
plt.subplot(2, 2, 3)
plt.title("原始灰度图的累积频率分布图")
plt.xlabel("像素强度")
plt.ylabel("累积频率")
plt.xlim([0, 255])
plt.plot(bins[:-1], cdf_normalized, color='blue')

# 绘制均衡化后灰度图的累积频率分布图
plt.subplot(2, 2, 4)
plt.title("均衡化后灰度图的累积频率分布图")
plt.xlabel("像素强度")
plt.ylabel("累积频率")
plt.xlim([0, 255])
plt.plot(equalized_bins[:-1], np.cumsum(equalized_hist) * 255 /
np.max(np.cumsum(equalized_hist)), color='red')

```

```
# 显示原始灰度图
plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.title("原始灰度图")
plt.imshow(gray_image, cmap='gray')
plt.axis('off')

# 显示均衡化后的灰度图
plt.subplot(1, 2, 2)
plt.title("均衡化后的灰度图")
plt.imshow(equalized_image, cmap='gray')
plt.axis('off')

plt.tight_layout()
plt.show()
```

9.1. 5) 基于“直方图均衡化”的图像效果差异

直方图均衡化是一种图像处理技术，用于改善图像的对比度，特别是当图像的可用数据由于背景或前景的光照不均匀而受到限制时，主要起到图像增强作用。主要体现在以下几个方面：

对比度提升：

均衡前：图像可能存在对比度不足的问题，特别是如果图像的像素值集中在灰度范围的某个区域，导致图像看起来较暗或较亮，缺乏细节。

均衡后：通过重新分配像素值，使得整个灰度范围得到更好的利用，图像的对比度得到提升，图像看起来更加清晰，细节更加丰富。

亮度调整：

均衡前：图像可能整体偏暗或偏亮，导致某些区域的细节丢失。

均衡后：图像的整体亮度得到调整，使得图像看起来更加均衡，不过于偏暗或偏亮。

细节增强：

均衡前：在光照不均匀的图像中，较暗或较亮的区域可能丢失细节。

均衡后：直方图均衡化通过调整像素值，使得原本在较暗或较亮区域的细节得以显现。

色彩分布：

均衡前：图像中的色彩分布可能不均匀，某些色彩可能过于集中。

均衡后：直方图均衡化使得色彩分布更加均匀，整个图像的色彩看起来更加自然和均衡。

视觉效果：

均衡前：图像可能看起来较为平淡，缺乏视觉冲击力。

均衡后：图像的视觉效果得到改善，看起来更加生动和吸引人。

然而，需要注意的是，直方图均衡化并不总是适用于所有类型的图像。例如，如果图像已经具有很好的对比度，或者图像的对比度问题是由于其他因素（如色彩偏差）引起的，那么直方图均衡化可能不会带来预期的效果，还会出现噪声等我们不愿意看到的情况。

此外，对于彩色图像，通常需要对每个颜色通道分别进行直方图均衡化，或者使用其他技术如直方图匹配来达到更好的效果。

应用方面：

生活中比较常见的应用在车牌识别中



如上图所示，在由于光线等条件不方便计算机视觉识别车牌信息时，使用直方图均衡化效果可见一斑，大幅提升了车牌识别的准确率。然而，它的效果不总是好的。



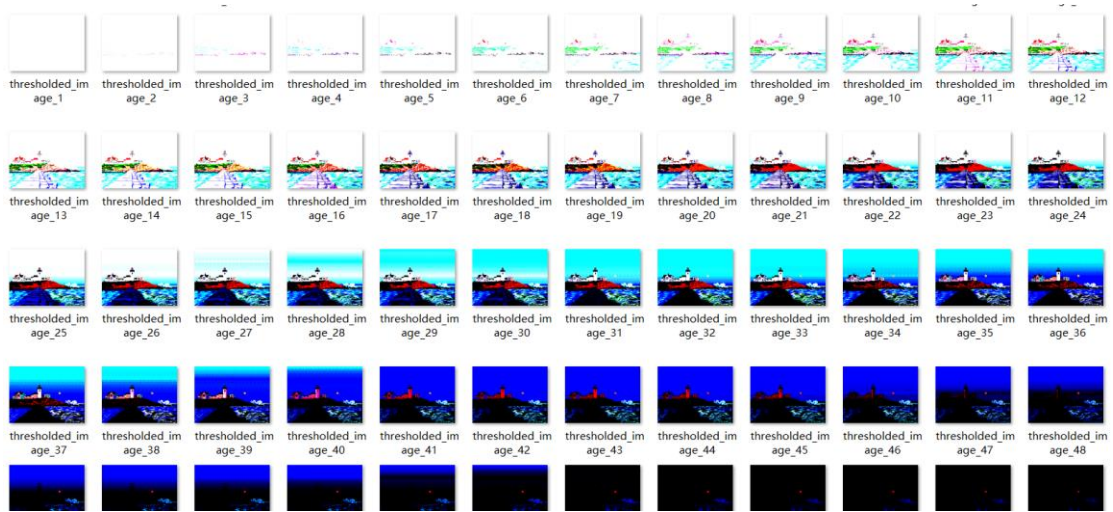
如上图所示，图像较为均衡时，使用直方图均衡化效果看会导致图片质量下降，反而会不利于甚至降低车牌识别的准确率。

9.2. 1)

编写程序实现了下图为原图的阈值处理结果系列图片：（完整代码在后面）



阈值处理结果系列图片缩略图如下图所示：



9.2. 2)

生成 gif 图如下图所示：



完整代码：

```
import cv2
import numpy as np
import imageio
import os

# 读取图像
image = cv2.imread('20201205151406_b3b55.jpg') # 读取彩色图像，不使用
cv2.IMREAD_GRAYSCALE

# 检查图像是否读取成功
if image is None:
    print("图像文件读取失败，请检查文件路径。")
else:
    # 设置起始阈值
    start_threshold = 2

    # 创建一个空列表来存储处理后的图像
    images = []

    # 循环设置阈值并处理图像
    threshold = start_threshold
    count = 0 # 用于保存文件时的编号
    while threshold <= 255:
        # 创建一个与原图像同样大小的全零矩阵，用于存放阈值处理后的结果
        thresholded_image = np.zeros_like(image)

        # 对每个通道应用阈值处理
        for i in range(3): # 遍历 BGR 三个通道
            _, channel = cv2.threshold(image[:, :, i], threshold, 255,
cv2.THRESH_BINARY)
            thresholded_image[:, :, i] = channel

        # 将处理后的图像添加到列表中
        images.append(thresholded_image)

        # 保存每张处理后的图像为 PNG 文件
        cv2.imwrite(f'thresholded_image_{count}.png', thresholded_image)
        count += 1

        # 更新阈值
        increment = start_threshold ** 2
        threshold += increment
```

```
# 检查是否达到 255
if threshold > 255:
    break

# 将图像列表保存为 GIF
imageio.mimsave('thresholded_images.gif', [np.array(image) for image in
images], 'GIF', duration=0.1)

print("阈值处理后的 GIF 图像已保存，且每张处理后的图片也已单独保存。")
```